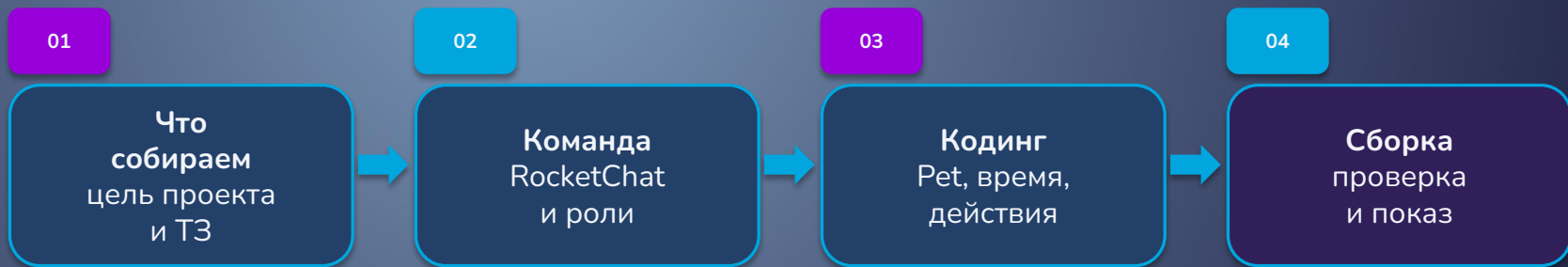


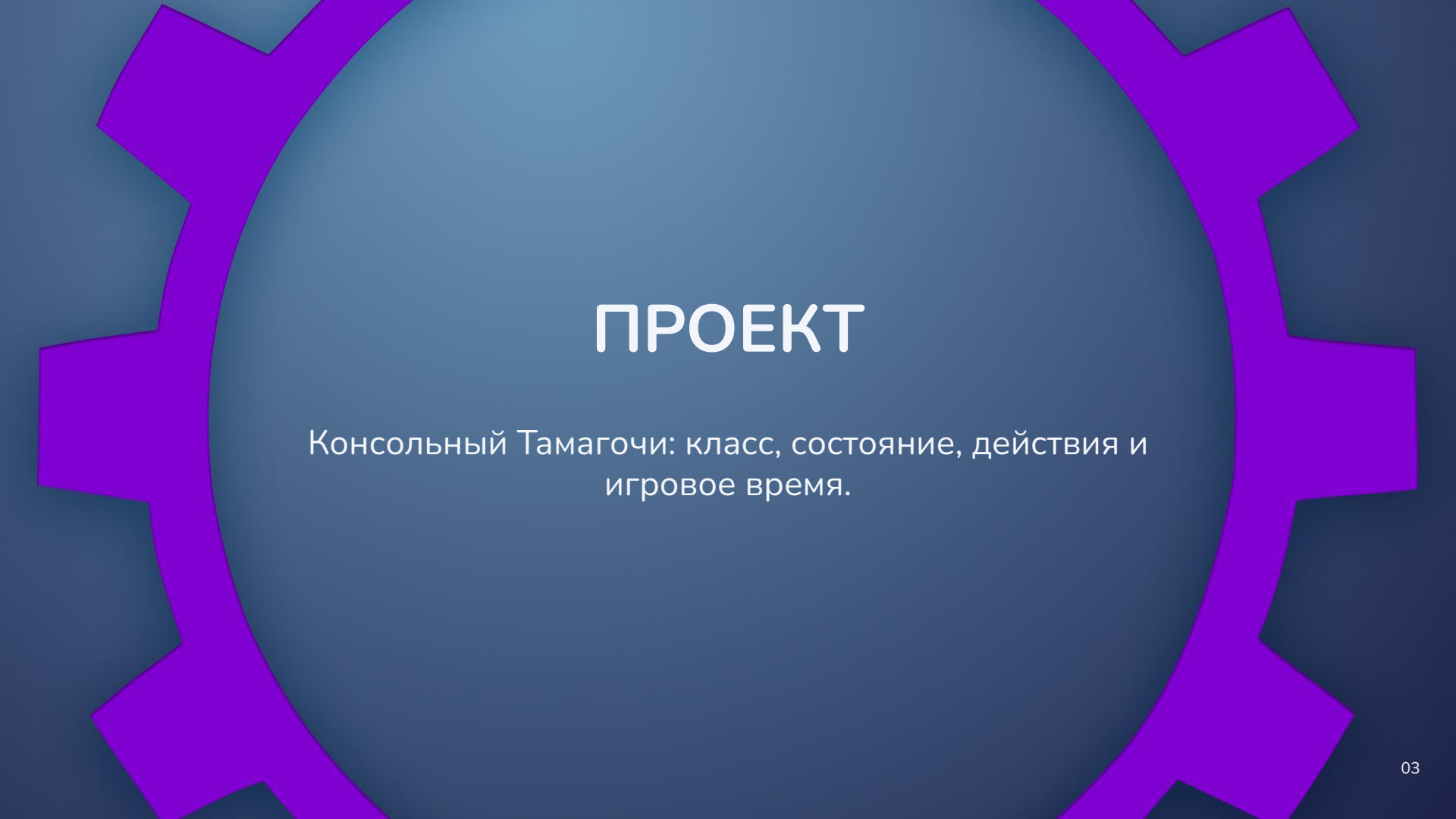


ПРОМЫШЛЕННОЕ ПРОГРАММИРОВАНИЕ

ПРОЕКТ: ТАМАГОЧИ
ЭТАП 1

Маршрут занятия





ПРОЕКТ

Консольный Тамагочи: класс, состояние, действия и игровое время.

Что собираем

Задачи

- виртуальный питомец
- кормление / игра / сон
- статус
- игровое время
- ошибки без падения

важно

Проект проверяется
как единая программа

ТЗ этапа 1

Состояние питомца

- name — имя
- hunger — голод
- mood — настроение
- energy — энергия
- все показатели: 0..10

Что умеет питомец

- feed()
- play()
- sleep()
- wake_up()
- показать состояние

Как работает время

- pass_time() двигает часы
- голод растёт, энергия падает
- ночью хочет спать
- сон бустит энергию

Основная структура класса

```
class Pet:
    def __init__(self, name): ...
    def feed(self): ...
    def play(self): ...
    def sleep(self): ...
    def wake_up(self): ...
    def pass_time(self, hours=1): ...
    def wants_to_sleep(self, hour):
...
    def __str__(self): ...
```

до кодинга

- имена методов
- параметры
- что возвращаем
- какие ошибки обрабатываем

Командные роли

A / B

питомец и действия

A — состояние Pet

B — feed(), play(), sleep(),
wake_up()

C / D

время и ошибки

C — игровое время

D — исключения и
проверки

E / F / G / H

меню и сборка

E — меню и основной цикл

F — тесты

G / H — сборка проекта

RocketChat: как передавать код

Фамилия Имя – роль В
Готово: `feed()`, `play()`
Файл: `actions.py`
Передано сборщику G
Комментарий: `play()` может вызвать `PetTooTiredError`

правила

- не уносим код в личку
- прикладываем файл
- пишем статус
- не меняем имена молча

Как работаем в команде

1

получить задачу

- понять, что получаем
- понять, что вернуть

2

написать код

- проверить запуск
- не ломать имена

3

передать дальше

- файл или ссылка
- статус в RocketChat

важно

RocketChat = общий журнал проекта

Задачи со звёздочкой

для быстрых

- добавить возраст
- добавить состояния «спит / бодрствует»
- сделать `status()`
- добавить пропуск нескольких часов

ещё идеи

- проверить крайние значения
- улучшить сообщения
- добавить случайность в одно действие
- проверить несколько ошибочных команд подряд

Проверка проекта

чек-лист

- Pet создаётся
- feed(), play(), sleep() работают
- значения остаются в 0..10
- время меняет состояние
- ошибки не завершают программу

```
def play(self):  
    raise NotImplementedError(  
        "play() ещё подключается"  
    )
```

важно

если не готово — ставим временную заглушку
и собираем проект дальше

Демонстрация и итог

показать

- создание питомца
- текущий статус
- два действия
- смену игрового времени
- обработку ошибки

Итоги занятия 04:

1. Что уже работает
2. Что осталось доделать
3. Кто за что отвечает
4. До какого срока